

Unit IV

Classification

BASIC CONCEPTS- Classification, Prediction, Issues regarding Classification and Prediction, Classification by Decision Tree Induction, Bayesian Classification-Bayes's Theorem, Naïve Bayesian Classification, Rule-Based Classification-Using IF-THEN Rules for Classification, Rule Extraction from a Decision Tree, Classification by Back propagation.

Part I

Classification is a model or a classifier, that is used to predict categorical labels.

Prediction is a model that is used to predict a continuous valued function or ordered value.

Data classification is a two-step process.

In the first step, a classifier is built describing a predetermined set of data classes or concepts. This is the learning step or training phase, where a classification algorithm builds the classifier by analyzing or learning from a training set made up of database tuples and their corresponding class labels. Because, the class label of each training tuple is provided, this step is also known as supervised learning. In the second step, the model is used for classification.

Issues Regarding Classification and Prediction

- **Preparing the data for classification and prediction**
- **Comparing classification and prediction methods**

Preparing the data for classification and prediction

The following preprocessing steps may be applied to the data to help improve the accuracy, efficiency and scalability of the classification or prediction process.

Data Cleaning: this refers to the preprocessing of the data to remove or reduce noise and the treatment of missing values.

Relevance analysis: many of the attributes in the data may be redundant. Correlation analysis can be used to identify whether any two given attributes are statistically related. A database may also contain irrelevant attributes. Attribute subset selection can be used in these cases to find a reduced set of attributes. Hence relevance analysis, in the form of correlation analysis and attribute subset selection, can be used to detect attributes that do not contribute to the classification or prediction task.

Data transformation and reduction: the data may be transformed by normalization, particularly when neural networks or methods involving distance measurements are used in the learning step. The data can also be transformed by generalizing it to higher level concepts, using concept hierarchies. Data can also be reduced by many other methods like wavelet transformation and principal component analysis, discretization techniques like binning, histogram analysis and clustering.

Comparing classification and prediction methods

Classification and prediction methods can be compared and evaluated according to the following criteria:

Accuracy: the accuracy of the classifier refers to the ability of a given classifier to correctly predict the class label of new or previously unseen data. Similarly, the accuracy of a predictor refers to how well a given predictor can guess the value of the predicted attribute for new or previously unseen data.

Speed: this refers to the computational costs involved in generating and using the given classifier or predictor.

Robustness: this is the ability of the classifier or predictor to make correct predictions given noisy data or data with missing values.

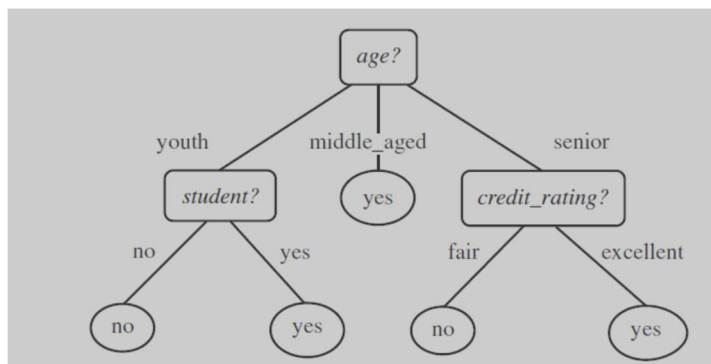
Scalability: this refers to the ability to construct the classifier or predictor efficiently given large amounts of data.

Interpretability: this refers to the level of understanding and insight that is provided by the classifier or predictor. It is subjective and therefore more difficult to assess.

Part II

Classification by Decision Tree Induction

Decision tree induction is the learning of decision trees from class labeled training tuples. A decision tree is like a flowchart like tree structure, where each internal node denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node holds a class label. The topmost node in a tree is the root node.



How are decision trees used for classification?

Given a tuple, X , for which the associated class label is unknown, the attribute values of the tuple are tested against the decision tree. A path is traced from the root to a leaf node, which holds the class predictions for that tuple. Decision trees can easily be converted to classification rules.

Decision Tree Induction

Decision tree induction algorithm is also known as ID3, Iterative Dichotomiser. C4.5 is a successor of ID3 algorithm. Classification and Regression Trees, CART, described the generation of binary decision trees. ID3, C4.5 and CART adopt a greedy nonbacktracking approach in which decision trees are constructed in a top-down recursive divide and conquer manner. The below is the basic algorithm for inducing a decision tree from training tuples.

Algorithm Generate_decision_tree – Generate a decision tree from the given training data.

Input: The training samples, samples, represented by discrete valued attributes; the set of candidate attribute list

Output: A decision tree

Method:

1. Create a node N;
2. If samples are all of the same class, C then
3. Return N as a leaf node labeled with the class C;
4. If attribute-list is empty then
5. Return N as a leaf node labeled with the most common class in samples; // majority voting
6. Select test-attribute, the attribute among attribute-list with the highest information gain;
7. Label node N with test-attribute;
8. For each known value a_i of test-attribute // partition the samples
9. Grow a branch from node N for the condition test-attribute = a_i ;
10. Let s_i be the set of samples in samples for which test-attribute = a_i ; // a partition
11. If s_i is empty then
12. Attach a leaf labeled with the most common class in sample;
13. Else attach the node returned by Generate_decision_tree (s_i , attribute-list – test-attribute)

The basic algorithm for decision tree induction is a greedy algorithm which constructs decision trees in a top-down recursive divide-and-conquer manner. The algorithm, summarized above, is a version of ID3, a well-known decision tree induction algorithm. The basic strategy is as follows:

- The tree starts as a single node representing the training samples (step 1).
- If the samples are all of the same class, then the node becomes a leaf and is labeled with that class (steps 2 and 3).
- Otherwise, the algorithm uses an entropy-based measure known as information gain as a heuristic for selecting the attribute that will best separate the samples into individual classes (step 6). This attribute becomes the “test” or “decision” attribute at the node (step 7). In this version of the algorithm, all attributes are categorical, i.e., discrete-valued. Continuous-valued attributes must be discretized.
- A branch is created for each known value of the test attribute, and the samples are partitioned accordingly (steps 8-10).
- The algorithm uses the same process recursively to form a decision tree for the samples at each partition. Once an attribute has occurred at a node, it need not be considered in any of the node's descendants (step 13).
- The recursive partitioning stops only when any one of the following conditions is true:
 1. All samples for a given node belong to the same class (step 2 and 3), or

2. There are no remaining attributes on which the samples may be further partitioned (step 4). In this case, majority voting is employed (step 5). This involves converting the given node into a leaf and labeling it with the class in majority among samples. Alternatively, the class distribution of the node samples may be stored; or
3. There are no samples for the branch test-attribute= a_i (step 11). In this case, a leaf is created with the majority class in samples (step 12).

Attribute selection measure: The information gain measure is used to select the test attribute at each node in the tree. Such a measure is referred to as an attribute selection measure or a measure of the goodness of split. The attribute with the highest information gain (or greatest entropy reduction) is chosen as the test attribute for the current node. This attribute minimizes the information needed to classify the samples in the resulting partitions and reflects the least randomness or "impurity" in these partitions. Such an information-theoretic approach minimizes the expected number of tests needed to classify an object and guarantees that a simple (but not necessarily the simplest) tree is found.

Let S be a set consisting of s data samples. Suppose the class label attribute has m distinct values defining m distinct classes, C_i (for $i = 1, \dots, m$). Let s_i be the number of samples of S in class C_i . The expected information needed to classify a given sample is given by:

$$I(s_1, s_2, \dots, s_m) = - \sum_{i=1}^m p_i \log_2(p_i)$$

where p_i is the probability that an arbitrary sample belongs to class C_i and is estimated by s_i/s . Note that a log function to the base 2 is used since the information is encoded in bits.

Let attribute A have v distinct values, $\{a_1, a_2, \dots, a_v\}$. Attribute A can be used to partition S into v subsets, $\{S_1, S_2, \dots, S_v\}$, where S_j contains those samples in S that have value a_j of A . If A were selected as the test attribute (i.e., best attribute for splitting), then these subsets would correspond to the branches grown from the node containing the set S . Let s_{ij} be the number of samples of class C_i in a subset S_j . The entropy, or expected information based on the partitioning into subsets by A is given by:

$$E(A) = \sum_{j=1}^v \frac{s_{1j} + \dots + s_{mj}}{s} I(s_{1j}, \dots, s_{mj}).$$

The term $\sum_{j=1}^v \frac{s_{1j} + \dots + s_{mj}}{s}$ acts as the weight of the j^{th} subset and is the number of samples in the subset (i.e., having value a_j of A) divided by the total number of samples in S . The smaller the entropy value is, the greater the purity of the subset partitions. The encoding information that would be gained by branching on A is

$$\text{Gain}(A) = I(s_1, s_2, \dots, s_m) - E(A).$$

In other words, $\text{Gain}(A)$ is the expected reduction in entropy caused by knowing the value of attribute A . The algorithm computes the information gain of each attribute. The attribute with the highest information gain is chosen

as the test attribute for the given set S. A node is created and labeled with the attribute, branches are created for each value of the attribute, and the samples are partitioned accordingly.

Example for Decision Tree Induction: To construct a decision tree using ID3 Algorithm for the given database. The tree is to be constructed using Information Gain as the attribute selection measure to select the best attribute for split.

rid	age	income	student	credit rating	Class: buys computer
1	<30	high	no	fair	no
2	<30	high	no	excellent	no
3	30-40	high	no	fair	yes
4	>40	medium	no	fair	yes
5	>40	low	yes	fair	yes
6	>40	low	yes	excellent	no
7	30-40	low	yes	excellent	yes
8	<30	medium	no	fair	no
9	<30	low	yes	fair	yes
10	>40	medium	yes	fair	yes
11	<30	medium	yes	excellent	yes
12	30-40	medium	no	excellent	yes
13	30-40	high	yes	fair	yes
14	>40	medium	no	excellent	no

There are two distinct class labels, yes and no that is $m = 2$. Let C1 correspond to class yes and C2 correspond to class no. The total number of samples are 14, out of which 9 tuples have class label to be yes and 5 tuples to have class label to be no. we have to first calculate the expected information needed to classify a given sample.

Let s_1 be the number of samples whose class label is yes and s_2 be the number of samples whose class labels is no that is $s_1 = 9$ and $s_2 = 5$.

$$\begin{aligned}
 I(s_1, s_2) &= I(9, 5) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} \\
 &= -0.6482 * -0.6374 - 0.357 - 1.4855 \\
 &= 0.4090 + 0.5305 \\
 &= 0.940
 \end{aligned}$$

Next, we need to compute the entropy of each attribute. Start with the attribute age. We need to look at the distribution of yes and no samples for each value of age. We compute the expected information for each of these distributions.

For attribute age, it has 3 distinct values that is $j = 3$, $age = < 30$, $age = 30-40$, $age = > 40$

$i = 2$, with $i = 1$ is yes and $i = 2$ is no.

therefore, s_{ij} = number of samples with class label = i and attribute value = j

for $age = < 30$

s_{11} = number of samples with class label = yes and $age = < 30$; $s_{11} = 2$

s_{21} = number of samples with class label = no and $age = < 30$; $s_{21} = 3$

$$\begin{aligned}
 I(s_{11}, s_{21}) &= I(2, 3) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \\
 &= -0.4 * -1.322 - 0.6 * -0.737 \\
 &= 0.5288 + 0.4422 \\
 &= 0.971
 \end{aligned}$$

For $age = 31 - 40$

s_{12} = number of samples with class label = yes and $age = 31 - 40$; $s_{12} = 4$

s_{22} = number of samples with class label = no and $age = 310 - 40$; $s_{22} = 0$

$$\begin{aligned}
 I(s_{12}, s_{22}) &= I(4, 0) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} \\
 &= 0
 \end{aligned}$$

For $age = > 40$

s_{13} = number of samples with class label = yes and $age = > 40$; $s_{13} = 3$

s_{23} = number of samples with class label = no and $age = > 40$; $s_{23} = 2$

$$I(s_{13}, s_{23}) = I(3, 2) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5}$$

$$\begin{aligned}
&= -0.6 * -0.737 - 0.4 * -1.322 \\
&= 0.4422 + 0.5288 \\
&= 0.971
\end{aligned}$$

The expected information needed to classify a given sample if the samples are partitioned according to age, is:

$$\begin{aligned}
E(\text{age}) &= \frac{5}{14} I(s_{11}, s_{21}) + \frac{4}{14} I(s_{12}, s_{22}) + \frac{5}{14} I(s_{13}, s_{23}) \\
&= \frac{5}{14} * 0.971 + \frac{4}{14} * 0 + \frac{5}{14} * 0.971 \\
&= 0.3468 + 0 + 0.3468 \\
&= 0.694
\end{aligned}$$

Hence, the gain in information from such a partitioning would be:

$$\begin{aligned}
\text{Gain}(\text{age}) &= I(s_1, s_2) - E(\text{age}) \\
&= 0.940 - 0.694 \\
&= 0.246
\end{aligned}$$

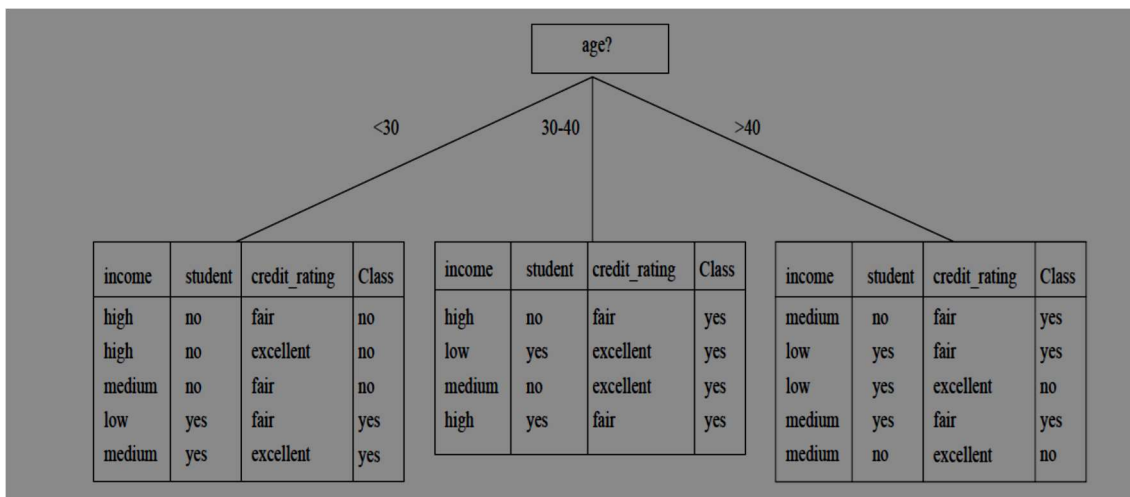
Similarly we have to calculate the information gain for other attributes like income, student and credit rating.

$$\text{Gain}(\text{income}) = 0.029$$

$$\text{Gain}(\text{student}) = 0.151$$

$$\text{Gain}(\text{credit rating}) = 0.048$$

Since age has the highest information gain among the attributes, it is selected as the test attribute. A node is created and labeled with age, and branches are grown for each of the attribute's values.



Gain Ratio

The information gain measure is biased toward tests with many outcomes. That is, it prefers to select attributes having a large number of values. For example, consider an attribute that acts as a unique identifier, such as `product_ID`. A split on `product_ID` would result in a large number of partitions, each one containing just one tuple. Because each partition is pure, the information required to classify data set D based on this partitioning would be $\text{Info}_{\text{product_ID}}(D) = 0$. Therefore the information gained by partitioning on this attribute is maximal. Clearly, such a partitioning is useless for classification.

C4.5, a successor of ID3, uses an extension to information gain known as gain ratio, which attempts to overcome this bias. It applies a kind of normalization to information gain using a “split information” value defined analogously with $\text{Info}(D)$ as

$$\text{splitInfo}_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \frac{|D_j|}{|D|}$$

This value represents the potential information generated by splitting the training data set D , into v partitions, corresponding to the v outcomes of a test on attribute A . Note that, for each outcome, it considers the number of tuples having that outcome with respect to the total number of tuples in D . It differs from information gain, which measures the information with respect to classification that is acquired based on the same partitioning. The gain ratio is defined as

$$\text{GainRatio} = \frac{\text{Gain}(A)}{\text{splitInfo}(A)}$$

The attribute with maximum gain ratio is selected as the splitting attribute. Note, however, that as the split information approaches 0, the ratio becomes unstable. A constraint is added to avoid this, whereby the information gain of the test selected must be large, at least as the average gain over all tests examined.

Computation of gain ratio for the attribute `income`. A test on `income` splits the data into three partitions, namely low, medium, and high, containing four, six, and four tuples, respectively. To compute the gain ratio of `income`, we first use Equation (6.5) to obtain

$$\begin{aligned} \text{SplitInfo}_A(D) &= -\frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) - \frac{6}{14} \times \log_2 \left(\frac{6}{14} \right) - \frac{4}{14} \times \log_2 \left(\frac{4}{14} \right). \\ &= 0.926. \end{aligned}$$

we have $\text{Gain}(\text{income}) = 0.029$. Therefore, $\text{GainRatio}(\text{income}) = 0.029/0.926 = 0.031$.

Gini index

The Gini index is used in CART. Using the notation described above, the Gini index measures the impurity of D , a data partition or set of training tuples, as

$$Gini(D) = 1 - \sum_{i=1}^m p_i^2,$$

where p_i is the probability that a tuple in D belongs to class C_i and is estimated by $|C_i, D|/|D|$. The sum is computed over m classes.

The Gini index considers a binary split for each attribute. Let's first consider the case where A is a discrete-valued attribute having v distinct values, $\{a_1, a_2, \dots, a_v\}$, occurring in D . To determine the best binary split on A , we examine all of the possible subsets that can be formed using known values of A . Each subset, S_A , can be considered as a binary test for attribute A of the form " $A \in S_A$?". Given a tuple, this test is satisfied if the value of A for the tuple is among the values listed in S_A . If A has v possible values, then there are 2^v possible subsets. For example, if income has three possible values, namely $\{\text{low}, \text{medium}, \text{high}\}$, then the possible subsets are $\{\text{low}, \text{medium}, \text{high}\}$, $\{\text{low}, \text{medium}\}$, $\{\text{low}, \text{high}\}$, $\{\text{medium}, \text{high}\}$, $\{\text{low}\}$, $\{\text{medium}\}$, $\{\text{high}\}$, and $\{\}$. We exclude the power set, $\{\text{low}, \text{medium}, \text{high}\}$, and the empty set from consideration since, conceptually, they do not represent a split. Therefore, there are $2^v - 2$ possible ways to form two partitions of the data, D , based on a binary split on A .

When considering a binary split, we compute a weighted sum of the impurity of each resulting partition. For example, if a binary split on A partitions D into D_1 and D_2 , the gini index of D given that partitioning is

$$Gini_A(D) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2).$$

The reduction in impurity that would be incurred by a binary split on a discrete- or continuous-valued attribute A is

$$\Delta Gini(A) = Gini(D) - Gini_A(D).$$

Induction of a decision tree using gini index. Let D be the training data of Table 6.1 where there are nine tuples belonging to the class buys computer = yes and the remaining five tuples belong to the class buys computer = no. A (root) node N is created for the tuples in D . We first use Equation (6.7) for Gini index to compute the impurity of D :

$$\begin{aligned} Gini(D) &= 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 \\ &= 0.459 \end{aligned}$$

To find the splitting criterion for the tuples in D , we need to compute the gini index for each attribute. Let's start with the attribute income and consider each of the possible splitting subsets. Consider the subset $\{\text{low}, \text{medium}\}$. This would result in 10 tuples in partition D_1 satisfying the condition " $\text{income} \in \{\text{low}, \text{medium}\}$." The remaining four tuples of D would be assigned to partition D_2 . The Gini index value computed based on this partitioning is

$$\begin{aligned}
& \frac{10}{14} \text{Gini}(D1) + \frac{4}{14} \text{Gini}(D2) \\
& \frac{10}{14} \left(\left(1 - \left(\frac{7}{10} \right)^2 - \left(\frac{3}{10} \right)^2 \right) \right) + \frac{4}{14} \left(\left(1 - \left(\frac{2}{4} \right)^2 - \left(\frac{2}{4} \right)^2 \right) \right) \\
& = 0.443 \\
& = \text{Gini}_{\text{income} \in \{\text{high}\}}(D)
\end{aligned}$$

Similarly, the Gini Index values for splits on the remaining subsets are 0.458, for the subsets {low, high} and {medium} and 0.450, for the subsets {medium, high} and {low}. Therefore, the best binary split for the attribute income is on {low, medium} or {high} because it minimizes the Gini index. Evaluating age, we obtain {<30, >40} or {30-40} as the best split for age with a Gini index of 0.375; the attributes student and credit rating are both binary, with Gini index values of 0.367 and 0.429, respectively. The attribute age and splitting subset {<30, >40} therefore give the minimum Gini index overall, with a reduction in impurity of $0.459 - 0.357 = 0.102$.

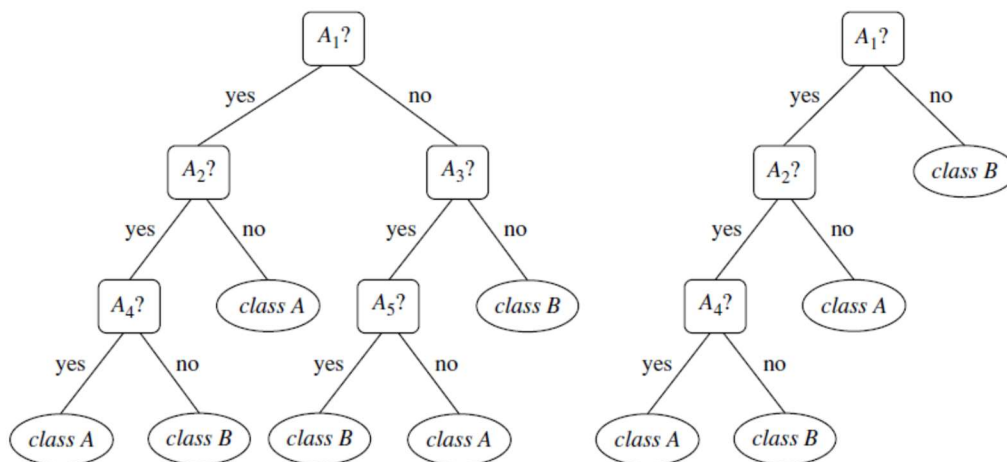
Tree Pruning

There are two common approaches to tree pruning. They are pre pruning and post pruning.

In the pre pruning approach, a tree is pruned by halting its construction early that is by deciding not to further split or partition the subset of training tuples at a given node. Upon halting, the node becomes a leaf. The leaf may hold the most frequent class among the subset tuples or the probability distribution of those nodes.

When constructing a tree, measures such as statistical significance, information gain, Gini index, and so on, can be used to assess the goodness of a split. If partitioning the tuples at a node would result in a split that falls below a prespecified threshold, then further partitioning of the given subset is halted. There are difficulties, however, in choosing an appropriate threshold. High thresholds could result in oversimplified trees, whereas low thresholds could result in very little simplification.

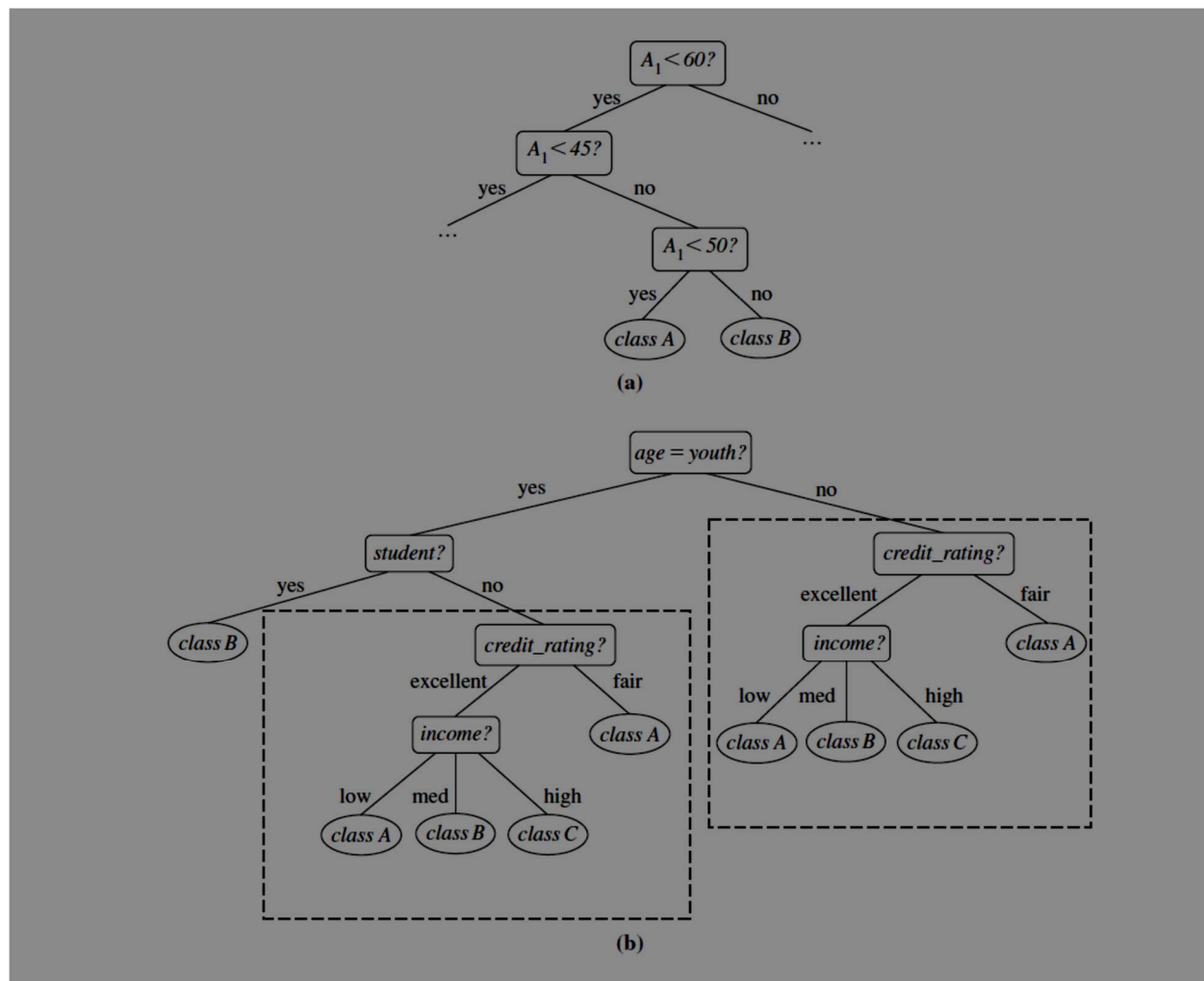
The second and more common approach is post pruning, which removes subtrees from a fully grown tree. A subtree at a given node is pruned by removing its branches and replacing it with a leaf. The leaf is labeled with the most frequent class among the subtree being replaced.



For example, notice the subtree at node “A3?” in the unpruned tree of Figure 8.6. Suppose that the most common class within this subtree is “class B.” In the pruned version of the tree, the subtree in question is pruned by replacing it with the leaf “class B.”

Alternatively, pre pruning and post pruning may be interleaved for a combined approach. Post pruning requires more computation than pre pruning, yet generally leads to a more reliable tree. No single pruning method has been found to be superior over all others.

Decision trees suffer from repetition and replication, making them over whelming to interpret. Repetition occurs when an attribute is repeatedly tested along a given branch of the tree. In replication, duplicate subtrees exist within the tree. These situations can impede the accuracy and comprehensibility of a decision tree.



- Subtree repetition, where an attribute is repeatedly tested along a given branch of the tree, like age
- Subtree replication, where duplicate subtrees exist within a tree, like the subtree headed by the node “credit rating”

Part III

Bayes Classification Methods

Bayesian Classifiers are statistical classifiers. They can predict class membership probabilities such as the probability that a given tuple belongs to a particular class.

Bayesian classification is based on Bayes Theorem.

Bayes Theorem

Bayes theorem is named after Thomas Bayes. Let X be a data tuple. In Bayesian terms, X is considered evidence. As usual, it is described by measurements made on a set of n attributes. Let H be some hypothesis such as that the data tuple X belongs to a specified class C . for classification problems, we want to determine $P(H/X)$, the probability that the hypothesis H holds given the evidence or observed data tuple, X . In other words, we are looking for the probability that tuple X belongs to class C , given that we know the attribute description of X .

$P(H/X)$ is called the posterior probability of H conditioned on X . In contrast, $P(H)$ is the prior probability of H .

$P(X/H)$ is called the posterior probability of X conditioned on H and $P(X)$ is called prior probability of X .

Example: suppose, the data tuples is confined to the customers described by the attributes age and income, respectively and that X is a 35-year-old customer with an income of 40,000. Suppose that H is the hypothesis that our customer will buy a computer. Then

$P(H/X)$ reflects the probability that customer X will buy a computer given that we know the customer's age and income.

$P(H)$ is the probability that any customer will buy a computer, regardless of age, income or any other information.

$P(X/H)$ is the probability that a customer, X , is 35 years old and earns 40,000, given that we know the customer will buy a computer.

$P(X)$ is the probability that a person from our set of customers is 35 years old and earns 40,000.

Now that the Bayes' theorem is defined as

$$P(H/X) = \frac{P(X/H) P(H)}{P(X)}$$

Naïve Bayesian Classification

The naïve Bayesian classifier or simple Bayesian classifier works as follows:

1. Let D be a training set of tuples and their associated class labels. As usual, each tuple is represented by an n -dimensional attribute vector, $X = \{x_1, x_2, \dots, x_n\}$, depicting n measurements made on the tuple from n attributes, respectively, $A_1, A_2, \dots, A_n$.
2. Suppose that there are m classes, $C_1, C_2, \dots, C_m$. Given a tuple, X , the classifier will predict that X belongs to the class having the highest posterior probability, conditioned on X . That is, the naïve Bayesian classifier predicts that tuple X belongs to the class C_i if and only if

$$P(C_i/X) > P(C_j/X) \text{ for } 1 \leq j \leq m, j \neq i$$

Thus we maximize $P(C_i/X)$. The class C_i for which $P(C_i/X)$ is maximized is called the maximum posteriori hypothesis. By Bayes' theorem,

$$P(C_i/X) = \frac{P(X/C_i) P(C_i)}{P(X)}$$

3. As $P(X)$ is constant for all classes, only $P(X/C_i) P(C_i)$ needs to be maximized. If the class prior probabilities are not known, then it is commonly assumed that the classes are equally likely, that is, $P(C_1) = P(C_2) = \dots = P(C_m)$, and we would therefore maximize $P(X/C_i)$. Otherwise, we maximize $P(X/C_i) P(C_i)$. Note that the class prior probabilities may be estimated by $P(C_i) = |C_{i,D}|/|D|$, where $|C_{i,D}|$ is the number of training tuples of class C_i in D .
4. Given data sets with many attributes, it would be extremely computationally expensive to compute $P(X/C_i)$. To reduce computation in evaluating $P(X/C_i)$. The naïve assumption of class-conditional independence is made. This presumes that the attributes' values are conditionally independent of one another, given the class label of the tuple (i.e., that there are no dependence relationships among the attributes). Thus,

$$\begin{aligned} P(X/C_i) &= \prod_{k=1}^n P(x_k/C_i) \\ &= P(x_1/C_i) \times P(x_2/C_i) \times \dots \times P(x_n/C_i) \end{aligned}$$

We can easily estimate the probabilities $P(x_1/C_i) \times P(x_2/C_i) \times \dots \times P(x_n/C_i)$ from the training tuples. Recall that here x_k refers to the value of attribute A_k for tuple X .

5. To predict the class label of X , $P(X/C_i) P(C_i)$ is evaluated for each class C_i . The classifier predicts that the class label of tuple X is the class C_i if and only if $P(X/C_i) P(C_i) > P(X/C_j) P(C_j)$ for $1 \leq j \leq m, j \neq i$. In other words, the predicted class label is the class C_i for which $P(X/C_i) P(C_i)$ is the maximum.

Problem: Suppose we wish to predict the class label of a tuple using naïve Bayesian classification, given the training data. The training data were described by the attributes age, income, student and credit rating. The class label attribute, buys_computer has two distinct values, namely yes and no. Let C_1 correspond to the class buys_computer = yes and C_2 corresponds to the class buys_computer = no.

Classify the following tuple: $X = (\text{age} = \text{youth}, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit rating} = \text{fair})$

Here we need to maximize $P(X/C_i) P(C_i)$ for $i = 1, 2$.

$P(C_i)$, the prior probability of each class, can be computed based on the training tuples.

$$P(\text{buys_computer} = \text{yes}) = 9/14 = 0.643$$

$$P(\text{buys_computer} = \text{no}) = 5/14 = 0.357$$

To compute $P(X/C_i)$ for $i = 1, 2$, we compute the following conditional probabilities:

$$P(\text{age} = \text{youth} \mid \text{buys computer} = \text{yes}) = 2/9 = 0.222$$

$$P(\text{income} = \text{medium} \mid \text{buys computer} = \text{yes}) = 4/9 = 0.444$$

$$P(\text{student} = \text{yes} \mid \text{buys computer} = \text{yes}) = 6/9 = 0.667$$

$$P(\text{credit rating} = \text{fair} \mid \text{buys computer} = \text{yes}) = 6/9 = 0.667$$

$$P(\text{age} = \text{youth} \mid \text{buys computer} = \text{no}) = 3/5 = 0.6$$

$$P(\text{income} = \text{medium} \mid \text{buys computer} = \text{no}) = 2/5 = 0.4$$

$$P(\text{student} = \text{yes} \mid \text{buys computer} = \text{no}) = 1/5 = 0.2$$

$$P(\text{credit rating} = \text{fair} \mid \text{buys computer} = \text{no}) = 2/5 = 0.4$$

Using these probabilities, we obtain

$$P(X \mid \text{buys_computer} = \text{yes}) = P(\text{age} = \text{youth} \mid \text{buys computer} = \text{yes}) \times P(\text{income} = \text{medium} \mid \text{buys computer} = \text{yes}) \\ \times P(\text{student} = \text{yes} \mid \text{buys computer} = \text{yes}) \times P(\text{credit rating} = \text{fair} \mid \text{buys computer} = \text{yes})$$

$$= 0.222 \times 0.444 \times 0.667 \times 0.667$$

$$= 0.044$$

$$P(X \mid \text{buys_computer} = \text{no}) = P(\text{age} = \text{youth} \mid \text{buys computer} = \text{no}) \times P(\text{income} = \text{medium} \mid \text{buys computer} = \text{no}) \times \\ P(\text{student} = \text{yes} \mid \text{buys computer} = \text{no}) \times P(\text{credit rating} = \text{fair} \mid \text{buys computer} = \text{no})$$

$$= 0.6 \times 0.4 \times 0.2 \times 0.4$$

$$= 0.019$$

To find the class, C_i , that maximizes $P(X/C_i) P(C_i)$, we compute

$$P(X \mid \text{buys_computer} = \text{yes}) \times P(\text{buys_computer} = \text{yes}) = 0.044 \times 0.643 = 0.028$$

$$P(X \mid \text{buys_computer} = \text{no}) \times P(\text{buys_computer} = \text{no}) = 0.019 \times 0.357 = 0.007$$

Therefore, the naïve Bayesian classifier predicts $\text{buys_computer} = \text{yes}$ for tuple X .

Part IV

Rule- Based Classification

Rule based classifiers are the type of classifier, where the learned model is represented as a set of IF- THEN rules.

Using IF- THEN Rules for Classification

Rules are a good way of representing information or bits of knowledge. A rule-based classifier uses a set of IF- THEN rules for classification. An IF- THEN rule is an expression of the form

IF *condition* THEN *conclusion*.

An example is rule R1.

R1: IF *age* = *youth* AND *student* = *yes* THEN *buys_computer* = *yes*.

The “IF” part of a rule is called rule antecedent or precondition. The “THEN” part is the rule consequent. In the rule antecedent, the condition consists of one or more attribute tests like *age* = *youth* and *student* = *yes*, that are logically ANDed. The rule consequent contains a class prediction. R1 can also be written as

R1: (*age* = *youth*) $\wedge$ (*student* = *yes*) $\rightarrow$ (*buys_computer* = *yes*)

If the condition in a rule antecedent holds true for a given tuple, we say that the rule antecedent is satisfied and that the rule **covers** the tuple. A rule R can be assessed by its coverage and accuracy. Given a tuple, X, from a class-labeled data set, D, let n_{covers} be the number of tuples covered by R; n_{correct} be the number of tuples correctly classified by R; and |D| be the number of tuples in D. we can define the coverage and accuracy of R as

$$\text{coverage}(R) = \frac{n_{\text{covers}}}{|D|}$$

$$\text{accuracy}(R) = \frac{n_{\text{correct}}}{n_{\text{covers}}}$$

that is, a rule’s coverage is the percentage of tuples that are covered by the rule. For a rule’s accuracy, we look at the tuples that it covers and see what percentage of them the rule can correctly classify.

If a rule is satisfied by X, the rule is said to be triggered. Suppose X is defined as follows.

X = (*age* = *youth*, *income* = *medium*, *student* = *yes*, *credit rating* = *fair*)

We would like to classify X according to *buys_computer*. X satisfies R1, which triggers the rule. If R1 is the only rule satisfied, then the rule fires by returning the class prediction for X. Triggering does not mean firing, because there may be more than one rule that is satisfied. If more than one rule is triggered, then we need a resolution strategy to figure out which rule gets to fire and assign its class prediction to X. There are many possible strategies. We look at two, namely *size ordering* and *rule ordering*.

The **size ordering** scheme assigns the highest priority to the rule that has toughest requirements, where toughness is measured by the rule antecedent size. That is, the triggering rule with the most attribute tests is fired.

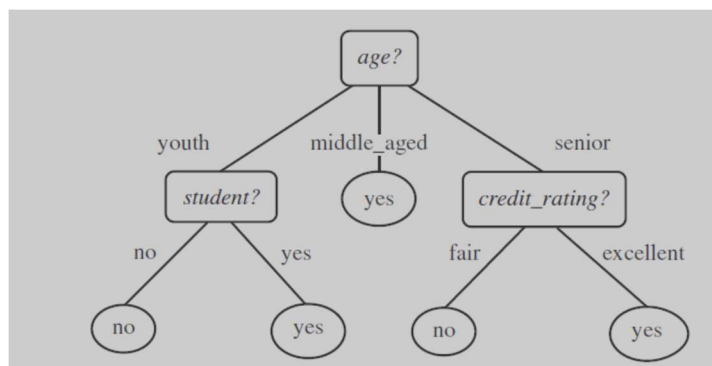
The **rule ordering** scheme priorities the rule beforehand. The ordering may be **class-based** or **rule-based**. With **class-based** ordering, the classes are sorted in order of decreasing importance of prevalence. That is, all the rules for the most prevalent(frequent) class come first, the rules for the next prevalent class comes next, and so on.

With **rule-based** ordering, the rules are organized into one long priority list, according to some measure of rule quality, such as accuracy, coverage, or size, or based on advice from domain experts. When rule ordering is used, the rule set is known as a **decision list**. With rule ordering, the triggering rule that appears earliest in the list has the highest priority, and so it gets to fire its class prediction. Any other rule that satisfies X is ignored.

When there is no rule to be satisfied by X, then a **default rule** can be set up, which can be used, based on training set.

Rule Extraction from a Decision Tree

To extract rules from a decision tree, one rule is created for each path from the root to a leaf node. Each splitting criterion along a given path is logically ANDed to form the rule antecedent ("IF" part). The lead node holds the class prediction, forming the rule consequent ("THEN" part).



- | | |
|---|--|
| R1: IF <i>age</i> = <i>youth</i> AND <i>student</i> = <i>no</i> | THEN <i>buys_computer</i> = <i>no</i> |
| R2: IF <i>age</i> = <i>youth</i> AND <i>student</i> = <i>yes</i> | THEN <i>buys_computer</i> = <i>yes</i> |
| R3: IF <i>age</i> = <i>middle_aged</i> | THEN <i>buys_computer</i> = <i>yes</i> |
| R4: IF <i>age</i> = <i>senior</i> AND <i>credit_rating</i> = <i>fair</i> | THEN <i>buys_computer</i> = <i>no</i> |
| R5: IF <i>age</i> = <i>senior</i> AND <i>credit_rating</i> = <i>excellent</i> | THEN <i>buys_computer</i> = <i>yes</i> |

Rule Induction using a Sequential Covering Algorithm

IF-THEN rules can be directly extracted from training data using a sequential covering algorithm. The name comes from the notion that the rules are learned sequentially (one at a time), where each rule for a given class will ideally cover many of the class tuples. There are many sequential covering algorithms. Popular variations include AQ, CN2 and the more recent RIPPER. The general strategy is as follows.

- Rules are learned one at a time.
- Each time a rule is learned, the tuples covered by the rule are removed, and the process repeats on the remaining tuples.
- This sequential learning of rules contrasts with decision tree induction. Because the path to each leaf in a decision tree corresponds to a rule, we can consider decision tree induction as learning a set of rules simultaneously.

Algorithm: Sequential covering. Learn a set of IF-THEN rules for classification.

Input:

- D , a data set of class-labeled tuples;
- Att_vals , the set of all attributes and their possible values.

Output: A set of IF-THEN rules.

Method:

```

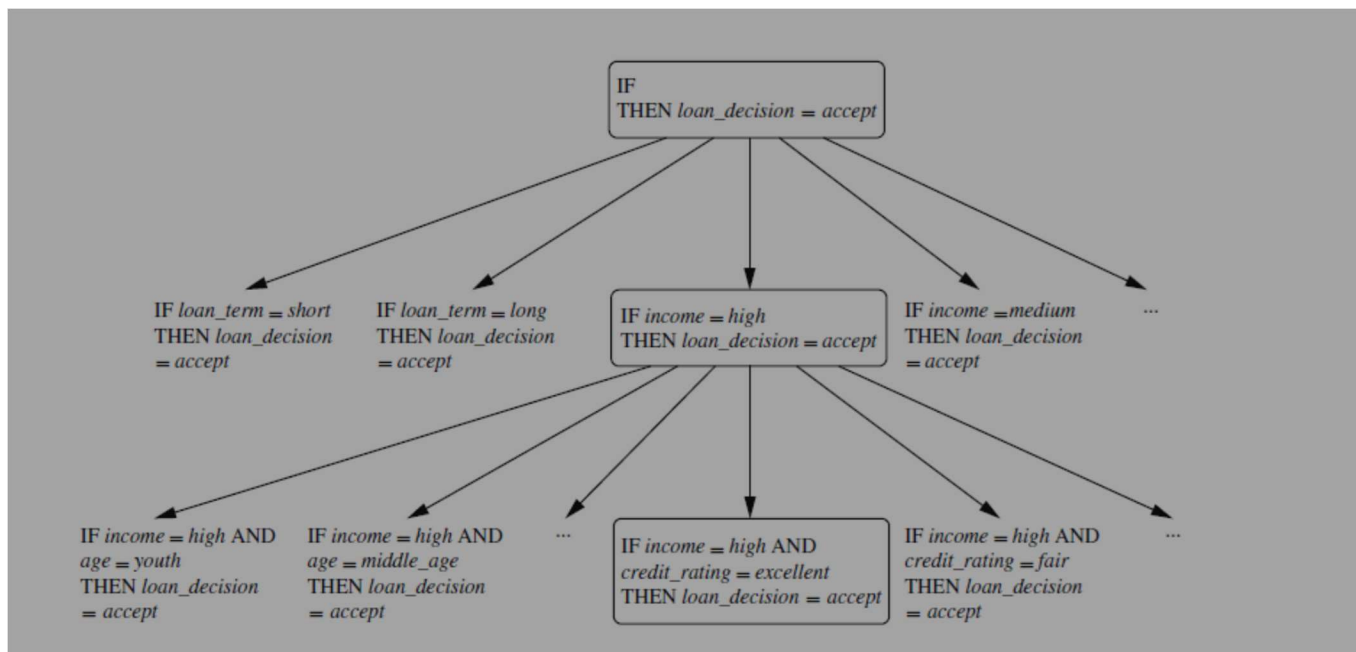
(1)  $Rule\_set = \{\}$ ; // initial set of rules learned is empty
(2) for each class  $c$  do
(3)   repeat
(4)      $Rule = \text{Learn\_One\_Rule}(D, Att\_vals, c)$ ;
(5)     remove tuples covered by  $Rule$  from  $D$ ;
(6)      $Rule\_set = Rule\_set + Rule$ ; // add new rule to rule set
(7)   until terminating condition;
(8) endfor
(9) return  $Rule\_Set$ ;

```

Basic Sequential Covering Algorithm

How are rules learned?

“How are rules learned?” Typically, rules are grown in a general-to-specific manner (Figure 8.11). We can think of this as a beam search, where we start off with an empty rule and then gradually keep appending attribute tests to it. We append by adding the attribute test as a logical conjunct to the existing condition of the rule antecedent. Suppose our training set, D , consists of loan application data. Attributes regarding each applicant include their age, income, education level, residence, credit rating, and the term of the loan. The classifying attribute is loan decision, which indicates whether a loan is accepted (considered safe) or rejected (considered risky).



Rule Quality Measures

In data mining, FOIL (First Order Inductive Learning) is a rule quality measure used to evaluate how good a rule is at distinguishing positive examples from negative ones. It comes from the FOIL algorithm, originally designed for learning logical rules.

FOIL measures how much a rule improves when you add a new condition to it. Does adding a new condition make my rule better at predicting the target class?"

The FOIL gain is given by:

$$\text{FOIL Gain} = p \left(\log_2 \frac{p}{p+n} \right) - \left(\log_2 \frac{P}{P+N} \right)$$

where

P = total positive examples before adding condition

N = total negative examples before adding condition

p = positive examples after adding condition

n = negative examples after adding condition

The term $\log_2 \frac{p}{p+n}$ is the accuracy after adding condition and $\log_2 \frac{P}{P+N}$ is the accuracy before adding condition.

Example:

Suppose we write a rule like

$$\text{student} = \text{yes} \rightarrow \text{buys_computer} = \text{yes}$$

Before adding the condition, there are 9 tuples whose class label is yes and 5 tuples whose class label is no, that is

$$P = 9$$

$$N = 5$$

After adding the condition, there are 6 tuples whose class label is yes and 1 tuple whose class label is no, that is

$$p = 6$$

$$n = 1$$

now, before accuracy is $9/9 + 5 = 0.642$ and after accuracy is $6/6 + 1 = 0.857$

clearly, after adding the condition, the accuracy of the rule is increased.

FOIL helps in the construction of rules step by step. It avoids adding useless conditions and balances accuracy and coverage.

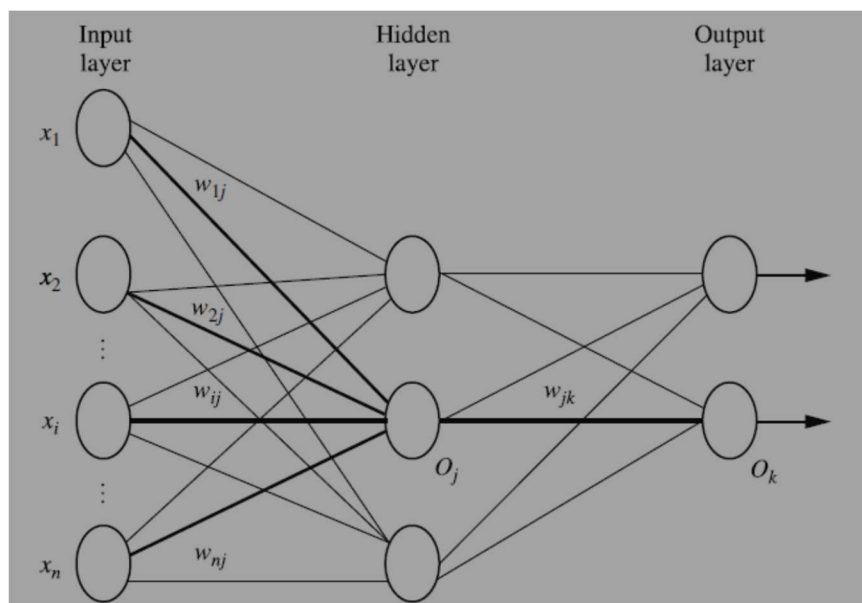
Part V

Classification by Backpropagation

Backpropagation is a neural network learning algorithm. There are many kinds of neural networks and neural network algorithms. The most popular neural network algorithm is backpropagation, which gained reputation in the 1980s.

A multi-layer Feed Forward Neural Network

A multilayer feed-forward neural network consists of an input layer, one or more hidden layers, and an output layer.



Each layer is made up of units. The inputs are fed simultaneously into the units making up the **input layer**. These inputs pass through the input layer and are then weighted and fed simultaneously to a second layer of “neuronlike” units, known as a **hidden layer**. The outputs of the hidden layer units can be input to another hidden layer, and so on. The weighted outputs of the last hidden layer are input to units making up **the output layer**, which emits the network’s prediction for given tuples. The units in the input layer are called **input units**. The units in the hidden layers and output layer are referred to as **output units**. The multilayer neural network shown in the above figure has two layers of output units. Therefore, we say that it is a two-layer neural network. Similarly, a network containing two hidden layers is called a three-layer neural network, and so on. It is a feed-forward network since none of the weights cycles back to an input unit or to a previous layer’s output unit. It is fully connected in that each unit provides input to each unit in the next forward layer.

Each output unit takes, as input, a weighted sum of the outputs from units in the previous layer. It applies a nonlinear (activation) function to the weighted input. Multilayer feed-forward neural networks are able to model the class prediction as a nonlinear combination of the inputs. From a statistical point of view, they perform nonlinear regression. Multilayer feed-forward networks, given enough hidden units and enough training samples, can closely approximate any function.

Backpropagation

Backpropagation learns by iteratively processing a data set of training tuples, comparing the network’s prediction for each tuple with the actual known target value. The target value may be the known class label of the training tuple (for classification problems) or a continuous value (for numeric prediction). For each training tuple, the weights are modified so as to minimize the mean-squared error between the network’s prediction and the actual target value. These modifications are made in the “backwards” direction (i.e., from the output layer) through each hidden layer down to the first hidden layer (hence the name backpropagation). Although it is not guaranteed, in general the weights will eventually converge, and the learning process stops. The steps involved are expressed in terms of inputs, outputs, and errors.

Initialize the weights: The weights in the network are initialized to small random numbers (e.g., ranging from -1.0 to 1.0, or -0.5 to 0.5). Each unit has a bias associated with it, as explained later. The biases are similarly initialized to small random numbers. Each training tuple, X , is processed by the following steps.

Propagate the inputs forward: First, the training tuple is fed to the network’s input layer. The inputs pass through the input units, unchanged. That is, for an input unit, j , its output, O_j , is equal to its input value, I_j . Next, the net input and output of each unit in the hidden and output layers are computed. The net input to a unit in the hidden or output layers is computed as a linear combination of its inputs. To compute the net input to the unit, each input connected to the unit is multiplied by its corresponding weight, and this is summed. Given a unit, j in a hidden or output layer, the net input, I_j , to unit j is

$$I_j = \sum_i w_{ij} O_i + \theta_j$$

Where w_{ij} is the weight of the connection from unit i in the previous layer to unit j ; O_i is the output of unit i from the previous layer; and θ_j is the bias of the unit. The bias acts as a threshold in that it serves to vary the activity of the unit.

Each unit in the hidden and output layers takes its net input and then applies an activation function to it, as illustrated in Figure 9.4. The function symbolizes the activation of the neuron represented by the unit. The logistic, or sigmoid, function is used. Given the net input I_j to unit j , then O_j , the output of unit j , is computed as

$$O_j = \frac{1}{1 + e^{-I_j}}$$

This function is also referred to as a squashing function, because it maps a large input domain onto the smaller range of 0 to 1. The logistic function is nonlinear and differentiable, allowing the backpropagation algorithm to model classification problems that are linearly inseparable. We compute the output values, O_j , for each hidden layer, up to and including the output layer, which gives the network's prediction.

Backpropagate the error: The error is propagated backward by updating the weights and biases to reflect the error of the network's prediction. For a unit j in the output layer, the error Err_j is computed by

$$Err_j = O_j (1 - O_j) (T_j - O_j)$$

where O_j is the actual output of unit j , and T_j is the known target value of the given training tuple. To compute the error of a hidden layer unit j , the weighted sum of the errors of the units connected to unit j in the next layer are considered. The error of a hidden layer unit j is

$$Err_j = O_j (1 - O_j) \sum_k Err_k w_{jk}$$

where w_{jk} is the weight of the connection from unit j to a unit k in the next higher layer, and Err_k is the error of unit k . The weights and biases are updated to reflect the propagated errors. Weights are updated by the following equations, where Δw_{ij} is the change in weight w_{ij} :

$$\Delta w_{ij} = (\eta) Err_j O_i$$

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

The variable η is the learning rate, a constant typically having a value between 0.0 and 1.0.

Biases are updated by the following equations, where $\Delta \theta_j$ is the change in bias θ_j

$$\Delta \theta_j = (\eta) Err_j$$

$$\theta_j = \theta_j + \Delta \theta_j$$

Algorithm: Backpropagation. Neural network learning for classification or numeric prediction, using the backpropagation algorithm.

Input:

- D , a data set consisting of the training tuples and their associated target values;
- l , the learning rate;
- $network$, a multilayer feed-forward network.

Output: A trained neural network.

Method:

- (1) Initialize all weights and biases in $network$;
- (2) **while** terminating condition is not satisfied {
- (3) **for** each training tuple X in D {
- (4) // Propagate the inputs forward:
- (5) **for** each input layer unit j {
- (6) $O_j = I_j$; // output of an input unit is its actual input value
- (7) **for** each hidden or output layer unit j {
- (8) $I_j = \sum_i w_{ij} O_i + \theta_j$; // compute the net input of unit j with respect to the previous layer, i
- (9) $O_j = \frac{1}{1 + e^{-I_j}}$; } // compute the output of each unit j
- (10) // Backpropagate the errors:
- (11) **for** each unit j in the output layer
- (12) $Err_j = O_j(1 - O_j)(T_j - O_j)$; // compute the error
- (13) **for** each unit j in the hidden layers, from the last to the first hidden layer
- (14) $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$; // compute the error with respect to the next higher layer, k
- (15) **for** each weight w_{ij} in $network$ {
- (16) $\Delta w_{ij} = (l) Err_j O_i$; // weight increment
- (17) $w_{ij} = w_{ij} + \Delta w_{ij}$; } // weight update
- (18) **for** each bias θ_j in $network$ {
- (19) $\Delta \theta_j = (l) Err_j$; // bias increment
- (20) $\theta_j = \theta_j + \Delta \theta_j$; } // bias update
- (21) } }